

Globalization and Localization

In this chapter, you'll learn the basics of how MySQL handles different character sets and languages.

Understanding Character Sets and Collation Sequences

Database tables are used to store and retrieve data. Different languages and character sets need to be stored and retrieved differently. Therefore, MySQL needs to accommodate different character sets (different alphabets and characters) as well as different ways to sort and retrieve data.

When discussing multiple languages and characters sets, you will run into the following important terms:

- **Character set:** A collection of letters and symbols
- **Encoding:** The internal representation of the members of a character set
- **Collation:** Instructions that dictate how characters are to be compared

Note

Why Collations Are Important Sorting text in English is easy, right? Well, maybe not. Consider the words *APE*, *apex*, and *Apple*. Are they in the correct sorted order? It depends on whether you want a case-sensitive sorting or a sorting that is not case-sensitive. The words would be sorted one way using a case-sensitive collation and another way for a collation that isn't case-sensitive. And this affects more than just sorting (as in data sorted using `ORDER BY`); it also affects searches (whether or not a `WHERE` clause looking for *apple* finds *APPLE*, for example). The situation gets even more complex when characters such as the French à or German ö are used, and it becomes even more complex when non-Latin-based character sets are used (Japanese, Hebrew, Russian, and so on).

In MySQL there is not much to worry about during regular database activity (SELECT, INSERT, and so forth). Rather, the decisions about which character set and collation to use occur at the server, database, and table level.

Working with Character Sets and Collation Sequences

MySQL supports a vast number of character sets. To see the complete list of supported character sets, use this statement:

► Input

```
| SHOW CHARACTER SET;
```

► Analysis

This statement displays all available character sets, along with a description of each one and the default collation for each one.

To see the complete list of supported collations, use this statement:

► Input

```
| SHOW COLLATION;
```

► Analysis

This statement displays all available collations, along with the character sets to which each one applies. Several character sets have more than one collation. `latin1`, for example, has several character sets for different European languages, and many sets appear twice: They appear once case-sensitive (designated by `_cs`) and once not case-sensitive (designated by `_ci`).

A default character set and collation are defined (usually by the system administration at installation time). In addition, when databases are created, default character sets and collations may be specified, too. To determine the character sets and collations in use, use these statements:

► Input

```
| SHOW VARIABLES LIKE 'character%';  
| SHOW VARIABLES LIKE 'collation%';
```

In practice, character sets can seldom be server-wide (or even database-wide) settings. Different tables, and even different columns, may require different character sets, and so both may be specified when a table is created.

To specify a character set and collation for a table, you use `CREATE TABLE` (seen in Chapter 21, “Creating and Manipulating Tables”) with additional clauses:

► Input

```
| CREATE TABLE mytable  
| (  
|
```

```

column1 INT,
column2 VARCHAR(10)
) DEFAULT CHARACTER SET hebrew
  COLLATE hebrew_general_ci;

```

► Analysis

This statement creates a two-column table and specifies both a character set and a collate sequence.

In this example, both `CHARACTER SET` and `COLLATE` are specified, but if only one of them (or neither of them) is specified, this is how MySQL determines what to use:

- If both `CHARACTER SET` and `COLLATE` are specified, those values are used.
- If only `CHARACTER SET` is specified, it is used along with the default collation for that character set (as specified in the `SHOW CHARACTER SET` results).
- If neither `CHARACTER SET` nor `COLLATE` is specified, the database default is used.

In addition to being able to specify character set and collation table-wide, MySQL also allows them to be set per column, as seen here:

► Input

```

CREATE TABLE mytable
(
  column1 INT,
  column2 VARCHAR(10),
  column3 VARCHAR(10) CHARACTER SET latin1
                                COLLATE latin1_general_ci
) DEFAULT CHARACTER SET hebrew
  COLLATE hebrew_general_ci;

```

► Analysis

Here `CHARACTER SET` and `COLLATE` are specified for the entire table as well as for a specific column.

As mentioned previously, the collation plays a key role in sorting data that is retrieved with an `ORDER BY` clause. If you need to sort specific `SELECT` statements by using a collation sequence other than the one used at table creation time, you may do so in the `SELECT` statement:

► Input

```

SELECT * FROM customers
ORDER BY lastname, firstname
  COLLATE latin1_general_cs;

```

► Analysis

This `SELECT` uses `COLLATE` to specify an alternate collation sequence (in this example, a case-sensitive one). This will obviously affect the order in which results are sorted.

Tip

Occasional Case-Sensitivity The `SELECT` statement just shown demonstrates a useful technique for performing case-sensitive searches on a table that is usually not case-sensitive. And, of course, the reverse works just as well.

Note

Other `SELECT COLLATE` Clauses In addition to being used in `ORDER BY` clauses, as shown here, `COLLATE` can be used with `GROUP BY`, `HAVING`, aggregate functions, aliases, and more.

One final point worth noting is that strings may be converted between character sets if absolutely needed. To do this, use the `Cast()` or `Convert()` functions.

Summary

In this chapter, you learned the basics of character sets and collations. You also learned how to define the character sets and collations for specific tables and columns and how to use alternate collations when needed.